

Práctica 6 - Capa de Transporte (Parte 2)

1. ¿Cuál es el puerto por defecto que se utiliza en los siguientes servicios? Web / SSH/ DNS/WebSeguro / POP3 / IMAP / SMTP Investigue en qué lugar en Linux y en Windows está descrita la asociación utilizada por defecto para cada servicio.

Servicio	Protocolo de Capa de Aplicación	Puerto por Defecto
Web (HTTP)	HTTP	80
SSH (Secure Shell)	Conexión remota segura a través de línea de comandos.	22
DNS (Sistema de Nombres de Dominio)	Resolución de nombres de dominio (UDP para consultas, TCP para transferencias de zona).	53
Web Seguro (HTTPS)	HTTP sobre SSL/TLS, cifrado	443
POP3 (Protocolo de Oficina de Correos v3)	POP3	110
IMAP (Protocolo de Acceso a Mensajes de Internet)	Recepción de correo con acceso remoto a carpetas	143
SMTP (Protocolo Simple de Transferencia de Correo)	SMTP	25 (Puerto 25 usado para la conexión Telnet al servidor SMTP)

2. Investigue qué es multicast. ¿Sobre cuál de los protocolos de capa de transporte funciona? ¿Se podría adaptar para que funcione sobre el otro protocolo de capa de transporte? ¿Por qué?

El *multicast* (o multidifusión) es la transferencia de datos desde un único emisor hacia muchos receptores en una sola operación de envío. Funciona tanto sobre UDP como sobre TCP.

3. Investigue cómo funciona el protocolo de aplicación FTP teniendo en cuenta las diferencias en su funcionamiento cuando se utiliza el modo activo de cuando se utiliza el modo pasivo ¿En qué se diferencian estos tipos de comunicaciones del resto de los protocolos de aplicación vistos?

FTP (File Transfer Protocol) es un protocolo de **aplicación** que permite **transferir archivos** entre un cliente y un servidor.

- **Utiliza dos conexiones TCP distintas:**

1. **Canal de control:** para comandos y respuestas.

- Puerto del servidor: **21/tcp**.

2. **Canal de datos:** para transferir archivos (subida, descarga, listado de directorios, etc.).

- Puerto varía según el **modo (activo o pasivo)**.

FTP modo activo (Active Mode)

♦ Funcionamiento:

1. El **cliente** se conecta al **servidor FTP** por el **puerto 21** (canal de control).
2. Cuando el cliente necesita transferir datos (listar archivos, descargar o subir), envía el comando **PORT** al servidor, indicando:
 - Su **dirección IP**
 - Y un **puerto aleatorio (>1023)** donde escuchará la conexión de datos.
3. El **servidor** abre una conexión **desde su puerto 20** hacia el puerto que el cliente indicó. Esa conexión se usa como **canal de datos**.

♦ Ejemplo:

- Canal de control: Cliente(>1023) → Servidor(21)
- Canal de datos: Servidor(20) → Cliente(>1023)

♦ Problema:

- Si el cliente está detrás de un **firewall o router NAT**, **no acepta conexiones entrantes**, y el servidor **no puede conectarse** al cliente.
Por eso el modo activo **falla en muchas redes modernas**.

⚙ FTP modo pasivo (Passive Mode)

♦ Funcionamiento:

1. El **cliente** se conecta al **servidor FTP** por el **puerto 21** (canal de control).
2. El cliente envía el comando **PASV** al servidor.
3. El **servidor responde** con:
 - Su **dirección IP**
 - Y un **puerto aleatorio (>1023)** donde él escuchará la conexión de datos.
4. El **cliente** inicia una **segunda conexión TCP** desde un puerto aleatorio hacia ese puerto del servidor.

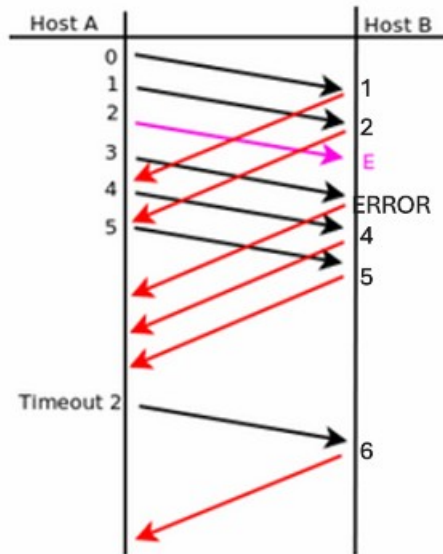
♦ Ejemplo:

- Canal de control: Cliente(>1023) → Servidor(21)
- Canal de datos: Cliente(>1023) → Servidor(>1023)

♦ Ventaja:

- Como **ambas conexiones las inicia el cliente**, **no hay problema con firewalls o NAT**.
Es el **modo más usado actualmente** (por ejemplo, en clientes como FileZilla o navegadores web).

4. Suponiendo Selective Repeat; tamaño de ventana 4 y sabiendo que E indica que el mensaje llegó con errores. Indique en el siguiente gráfico, la numeración de los ACK que el host B envía al Host A.



Selective Repeat es un protocolo de **control de errores y flujo** utilizado en la capa de enlace o transporte (como TCP en parte).

Permite **enviar varios frames (o segmentos)** sin esperar confirmación de cada uno, pero **sólo retransmite los que tienen error**.

La filosofía principal de la Repetición Selectiva es **evitar las retransmisiones innecesarias** que ocurren en protocolos como GBN. En lugar de retransmitir un gran número de paquetes (retroceder N) cuando se detecta una pérdida, SR solo retransmite aquellos paquetes que se sospecha que llegaron con errores o que se perdieron.

ACK significa **Acknowledgment** (confirmación o acuse de recibo).

Es un mensaje que **envía el receptor al emisor para avisar que recibió correctamente un paquete (o varios)**.

Entonces:

- **Tramas 0, 1 y 2** llegan bien → el receptor **envía ACK0, ACK1, ACK2**.
- **Trama 3** llega con error (E) → el receptor **no envía ACK3**.
- **Trama 4** llega correctamente → el receptor **envía ACK4**.
(En Selective Repeat, aunque haya un hueco, se pueden confirmar los que llegan bien).
- El emisor, al no recibir ACK3, **retransmite solo la 3** más adelante.

Mirá la numeración de los paquetes que llegan (0, 1, 2, 3, ...).

1. Cada paquete correcto → **ACK = número siguiente**.
2. Si un paquete llega con error (E) → **ese ACK falta**.
3. El emisor deduce cuál faltó mirando qué ACKs no llegaron.

→ el ACK “siguiente” se calcula como:

$$\text{ACK} = \text{número_de_paquete_recibido} + 1$$

5. ¿Qué restricción existe sobre el tamaño de ventanas en el protocolo Selective Repeat?

En **Selective Repeat**, tanto el **emisor** como el **receptor** manejan una **ventana de tamaño N**, que indica cuántos paquetes pueden enviarse o aceptarse **sin esperar confirmación**.

Pero hay un problema si la numeración de tramas **se reutiliza** (por ejemplo, cuando llega al final y vuelve a 0).

Ahí puede pasar que el receptor **confunda un paquete nuevo con uno viejo**.

En Selective Repeat, el tamaño de la **ventana de transmisión (N)** debe ser **como máximo la mitad del espacio total de numeración de tramas**.

Formalmente: $N < \text{Número total de secuencias posibles} / 2$

Porque si la ventana fuera más grande, un número de secuencia **podría estar activo en dos ventanas diferentes al mismo tiempo**:

- Una ventana vieja (de paquetes ya enviados pero pendientes de ACK)
- Y una ventana nueva (de paquetes nuevos que están siendo enviados)

Esto haría que el receptor **no pueda distinguir si una trama es nueva o una retransmisión**, generando **confusión y errores**.

NOTA: Una **trama (frame)** es la **unidad de información** que se **envía a nivel de enlace de datos** (Capa 2 del modelo OSI).

Es el **paquete completo** que **viaja entre dos nodos directamente conectados** (por ejemplo, dos computadoras unidas por un cable o un switch).

Una **trama** = *lo que se envía por la red física o enlace local (Capa 2)*.

Dentro de ella **viaja un paquete IP (Capa 3)**, que a su vez puede contener un **segmento TCP o UDP (Capa 4)**.

6. De acuerdo a la captura TCP de la siguiente figura, indique los valores de los campos **borroneados**.

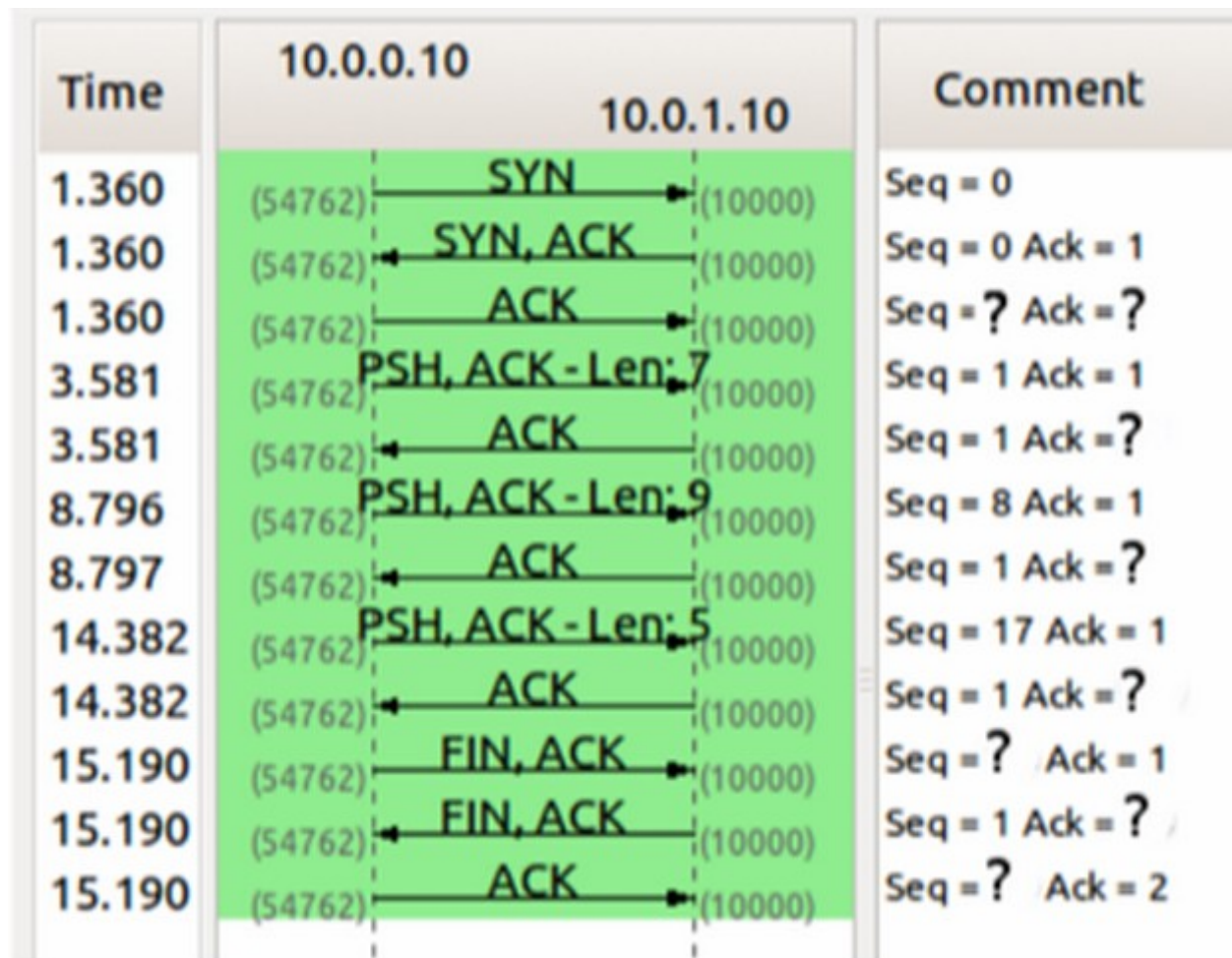
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.1.1	172.20.1.100	TCP	74	41749 > vce [] Seq= win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=270132 TSecr=0
2	0.001264	172.20.1.100	172.20.1.1	TCP	74	vce > 41749 [SYN, ACK] Seq=1047471501 Ack=3933822138 Win=5792 Len=0 MSS=1460 SACK_PERM=1
3	0.001341			TCP	66	> [] Seq= Ack= Win=5888 Len=0 TSval=270132 TSecr=1877442

Internet Protocol Version 4, Src: 172.20.1.100 (172.20.1.100), Dst: 172.20.1.1 (172.20.1.1)
Transmission Control Protocol, Src Port: vce (11111), Dst Port: 41749 (41749), Seq: 1047471501, Ack: 3933822138, Len: 0
Source port: vce (11111)
Destination port: 41749 (41749)
[Stream index: 0]
Sequence number: 1047471501
Acknowledgement number: 3933822138
Header length: 40 bytes
Flags: 0x012 (SYN, ACK)
000. = Reserved: Not set
...0 = Nonce: Not set
.... 0... = Congestion Window Reduced (CWR): Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...1 = Acknowledgement: Set
....0 = Push: Not set
....0 = Reset: Not set
*1 = Syn: Set
....0 = Fin: Not set
Window size value: 5792
[Calculated window size: 5792]
Checksum: 0x9803 [validation disabled]

El primer Seq = 3933822137

El Seq es uno menos que el ACK y después es el mismo. Los Flags son: SYN, SYN ACK, ACK

7. Dada la sesión TCP de la figura, completar los valores marcados con un signo de interrogación.



Seq	Ack
0	-
0	1
1	1
1	1
1	8
8	1
1	17
17	1
1	22
22	1
1	23
23	2

8. ¿Qué es el RTT y cómo se calcula? Investigue la opción TCP timestamp y los campos TSval y Tsecl.

RTT (Round Trip Time) = *Tiempo de ida y vuelta*

Es el tiempo que tarda un paquete en ir desde el origen al destino y volver con su respuesta (ACK).

Se mide en segundos o milisegundos, y varía según:

- La distancia física y cantidad de saltos intermedios.
- El estado de congestión de la red.
- El retardo de procesamiento en los hosts.

En TCP, el **RTT** se calcula automáticamente midiendo el tiempo entre:

- El envío de un **segmento con un número de secuencia X**, y
- La recepción del **ACK que confirma X**.

Ejemplo:

- El host envía un paquete Seq = 1 a las **3.581 s**.
- Recibe el ACK = 2 (confirmando ese paquete) a las **8.796 s**.
→ Entonces el **RTT $\approx 8.796 - 3.581 = 5.215$ s**.

En la práctica, TCP usa un promedio ponderado (SRTT) para estimar el RTT y ajustar el *timeout* de retransmisión (RTO).

Son **opciones TCP** que permiten medir el RTT de manera más precisa y mantener la conexión más eficiente.

♦ **TSval (Timestamp Value)**

- Es un **contador de tiempo local** del host que envía el segmento.
- Se incrementa continuamente con el tiempo (no es el reloj real, sino un contador relativo).

♦ **Tsecl (Timestamp Echo Reply)**

- Es el valor **TSval** recibido previamente del otro extremo, que el host **reenvía de vuelta** en su siguiente segmento.
- Permite que el emisor calcule fácilmente el RTT.

9. Para la captura tcp-captura.pcap, responder las siguientes preguntas.

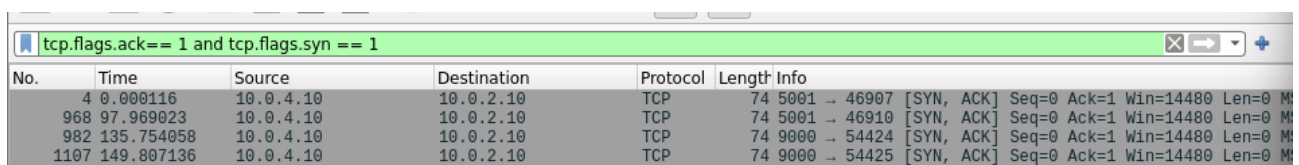
a. ¿Cuántos intentos de conexiones TCP hay?

Hay 10 intentos de conexiones TCP, es decir, 10 veces que se envió SYN (SYN == 1)

b. ¿Cuáles son la fuente y el destino (IP:port) para c/u?

Son entre las direcciones IP: 10.0.4.10 y 10.0.2.10

c. ¿Cuántas conexiones TCP exitosas hay en la captura? ¿Cómo diferencia las exitosas de las que no lo son? ¿Cuáles flags encuentra en cada una?



The image shows a Wireshark packet capture window with a filter set to 'tcp.flags.ack == 1 and tcp.flags.syn == 1'. The packet list shows four TCP segments:

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000116	10.0.4.10	10.0.2.10	TCP	74	5001 → 46907 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
968	97.969023	10.0.4.10	10.0.2.10	TCP	74	5001 → 46910 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
982	135.754058	10.0.4.10	10.0.2.10	TCP	74	9000 → 54424 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
1107	149.807136	10.0.4.10	10.0.2.10	TCP	74	9000 → 54425 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M

d. Dada la primera conexión exitosa responder:

i. ¿Quién inicia la conexión?

La inicia el Destination (10.0.2.10), porque es al que le están confirmando la conexión.

ii. ¿Quién es el servidor y quién el cliente?

El servidor es el 10.0.4.10 y el cliente el 10.0.2.10, quien inicia la conexión.

iii. ¿En qué segmentos se ve el 3-way handshake?

iv. ¿Cuáles ISNs se intercambian?

v. ¿Cuál MSS se negoció?

vi. ¿Cuál de los dos hosts envía la mayor cantidad de datos (IP:port)?

e. Identificar primer segmento de datos (origen, destino, tiempo, número de fila y número de secuencia TCP).

i. ¿Cuántos datos lleva?

ii. ¿Cuándo es confirmado (tiempo, número de fila y número de secuencia TCP)?

iii. La confirmación, ¿qué cantidad de bytes confirma?

f. ¿Quién inicia el cierre de la conexión? ¿Qué flags se utilizan? ¿En cuáles segmentos se ve (tiempo, número de fila y número de secuencia TCP)?

10. Responda las siguientes preguntas respecto del mecanismo de control de flujo.

a. ¿Quién lo activa? ¿De qué forma lo hace?

El receptor es quien activa el control de flujo.

Cómo lo hace:

- En cada segmento TCP que envía (normalmente los ACK), el receptor incluye un campo llamado **"Window Size"**.
- Ese valor indica **cuántos bytes más puede recibir** sin desbordar su buffer.

Ejemplo:

ACK = 201, Win = 5000

➔ Significa:

"Ya recibí todo hasta el byte 200, y puedo recibir 5000 bytes más antes de llenar mi buffer".

De esta forma, el **emisor regula su velocidad de envío** según la ventana que le anuncia el receptor.

b. ¿Qué problema resuelve?

Resuelve el problema de **desbordamiento (overflow)** del receptor.

Si el emisor envía datos más rápido de lo que el receptor puede procesar o almacenar, los buffers del receptor se llenan y se perderían datos.

El control de flujo evita eso, asegurando que el emisor **solo envíe lo que el receptor puede manejar**.

c. ¿Cuánto tiempo dura activo y qué situación lo desactiva?

El control de flujo está activo durante toda la conexión TCP,

pero su efecto **varía dinámicamente** según el tamaño de la ventana que anuncia el receptor.

- Si el receptor tiene poco espacio disponible, **reduce la ventana (Win ↓)** → el emisor envía menos.
- Si el receptor vacía su buffer (procesa datos), **aumenta la ventana (Win ↑)** → el emisor puede volver a enviar más.
- Si la ventana llega a **0**, el emisor **deja de enviar** (entra en "Zero Window").
Luego el receptor avisa cuando vuelve a tener espacio ("Window Update").

Por lo tanto, **nunca se "desactiva" del todo**, pero **puede suspender temporalmente el envío de datos** si la ventana llega a cero.

11. Responda las siguientes preguntas respecto del mecanismo de control de congestión.

a. ¿Quién activa el mecanismo de control de congestión? ¿Cuáles son los posibles disparadores?

El mecanismo de control de congestión lo activa el emisor TCP, no el receptor (a diferencia del control de flujo).

Se activa cuando el emisor detecta congestión en la red o la posibilidad de que ocurra.

Posibles disparadores (signos de congestión):

1. **Pérdida de segmentos** (por timeout o retransmisión).
→ Principal indicador de congestión.
2. **Duplicación de ACKs** (por ejemplo, recibir 3 ACKs iguales seguidos).
→ Señal de que un paquete se perdió.
3. **Incremento del RTT (Round Trip Time)**.
→ Puede indicar que los routers o enlaces están saturados.

b. ¿Qué problema resuelve?

Evita la sobrecarga en la red.

Sin este control, si muchos emisores enviaran datos al máximo de su velocidad:

- Los routers se llenarían.
- Se producirían **colas, pérdidas y retransmisiones masivas**.
- Esto generaría aún más congestión (efecto bola de nieve).

El control de congestión TCP adapta dinámicamente la tasa de envío según el estado de la red, manteniendo el tráfico fluido y evitando el colapso.

c. Diferencie slow start de congestion-avoidance.

Son dos fases del control de congestión en TCP.

Ambas usan una variable llamada Congestion Window (cwnd) que indica cuántos bytes puede tener “en vuelo” sin ser confirmados.

Slow Start (Arranque lento)

- Se activa **al iniciar una conexión o después de una pérdida grave**.
- **TCP no conoce aún la capacidad de la red**, así que empieza con prudencia:
 - $cwnd = 1$ MSS (una unidad de máximo segmento).
 - Por cada ACK recibido, **duplica cwnd** → crecimiento **exponencial**.
- Continúa hasta alcanzar un umbral (ssthresh) o detectar congestión.

Crecimiento:

1, 2, 4, 8, 16, ...

(en cantidad de MSS por RTT)

Es “lento” solo en nombre: en realidad crece rápido, hasta descubrir la capacidad máxima de la red.

Congestion Avoidance (Evitar congestión)

- Se entra en esta fase cuando `cwnd` alcanza el **umbral (`ssthresh`)**.
- Ahora el crecimiento pasa a ser **lineal**, no exponencial.
- Por cada RTT exitoso (sin pérdida):
$$cwnd = cwnd + 1 \text{ MSS}$$

Crecimiento:

17, 18, 19, 20, ...

Si hay pérdida de paquetes:

- $ssthresh = cwnd / 2$
- `cwnd` se reinicia (según el tipo de pérdida):
 - Timeout → vuelve a 1 MSS (*Slow Start de nuevo*).
 - Triple ACK duplicado → *Fast Retransmit* y *Fast Recovery*.

12. Para la captura `udp-captura.pcap`, responder las siguientes preguntas.

- ¿Cuántas comunicaciones (`srcIP,srcPort,dstIP,dstPort`) UDP hay en la captura?
- ¿Cómo se podrían identificar las exitosas de las que no lo son?
- ¿UDP puede utilizar el modelo cliente/servidor?
- ¿Qué servicios o aplicaciones suelen utilizar este protocolo? ¿Qué requerimientos tienen?
- ¿Qué hace el protocolo UDP en relación al control de errores?
- Con respecto a los puertos vistos en las capturas, ¿observa algo particular que lo diferencie de TCP?